

Installing Toolchains for Pico 2 for Windows 10 without using V.S. Code

This video is a step by step instruction on setting up the programming environment for the RP2040 processor on the Raspberry Pi Pico used in my videos.

These are the links to webpages used in this video:

ARM GCC toolchain: <https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>

Download this file: [arm-gnu-toolchain-14.2.rel1-mingw-w64-x86_64-arm-none-eabi.exe](https://developer.arm.com/-/media/Tools/GNU/arm-gnu-toolchain-14.2.rel1-mingw-w64-x86_64-arm-none-eabi.exe)

Risc-v pico toolchain: <https://github.com/raspberrypi/pico-sdk-tools/releases>

Download this file: [riscv-toolchain-14-x64-win.zip](https://github.com/raspberrypi/pico-sdk-tools/releases/download/v1.0.0/riscv-toolchain-14-x64-win.zip)

Python: <https://www.python.org/>

Install this file: <https://www.python.org/ftp/python/3.13.1/python-3.13.1-amd64.exe>

GCC for Windows: <https://winlibs.com/>

Download this file:

https://github.com/brechtsanders/winlibs_mingw/releases/download/14.2.0posix-19.1.1-12.0.0-ucrt-r2/winlibs-x86_64-posix-seh-gcc-14.2.0-mingw-w64ucrt-12.0.0-r2.7z

Make for Windows: <https://sourceforge.net/projects/gnuwin32/files/make/3.81/>

Download this file:

<https://sourceforge.net/projects/gnuwin32/files/make/3.81/make-3.81.exe/download>

CMake: <https://cmake.org/download/>

Download this file:

https://github.com/Kitware/CMake/releases/download/v3.31.4/cmake-3.31.4-windows-x86_64.msi

Visual Studio Code: <https://code.visualstudio.com/>

Download this file: <https://code.visualstudio.com/sha/download?build=stable&os=win32-x64-user>

Pico Examples: <https://github.com/raspberrypi/pico-examples>

Download this file: <https://github.com/raspberrypi/pico-examples/archive/refs/heads/master.zip>

Pico Extras: <https://github.com/raspberrypi/pico-extras>

Download this file: <https://github.com/raspberrypi/pico-extras/archive/refs/heads/master.zip>

Pico Playground: <https://github.com/raspberrypi/pico-playground>

Download this file: <https://github.com/raspberrypi/pico-playground/archive/refs/heads/master.zip>

Pico SDK - From the Developer Command Prompt:

- 1) Navigate into the Pico directory you created to hold the pico tools
- 2) Clone the SDK: `git clone https://github.com/raspberrypi/pico-sdk.git`

- 3) Navigate into the sdk directory: `cd ~/pico-sdk`
- 4) Update the submodule: `git submodule update --init`
- 5) Add/ Update the PICO_SDK_PATH in the user environment

Install Picotool:

- 1) Refer to the Building instructions here: <https://github.com/raspberrypi/picotool>
- 2) Run this command from the pico-sdk directory: `git submodule update --init lib/mbedtls`
- 3) Install libUSB: <https://github.com/libusb/libusb>
Download this file:
<https://github.com/libusb/libusb/releases/download/v1.0.27/libusb-1.0.27.7z>
- 4) Using 7zip, extract it into the tools directory
- 5) Add/ Update LIBUSB_ROOT path in the user environment
- 6) Download Picotool: <https://github.com/raspberrypi/picotool/archive/refs/heads/master.zip>
- 7) Extract picotool into a directory and create a build folder
- 8) Using Developer Command Prompt, navigate into that build folder
- 9) Execute: `cmake -DCMAKE_INSTALL_PREFIX=<YOUR_INSTALL_DIR> -DPICOTOOL_FLAT_INSTALL=1 ..`
- 10) Execute: `cmake --build .`
- 11) Execute: `cmake --install .`
- 12) Install USB Driver: <https://zadig.akeo.ie/>
Download this file:
<https://github.com/pbatard/libwdi/releases/download/v1.5.1/zadig-2.9.exe>
- 13) Execute Zadig and Install the driver
- 14) Add/ Update the user path to picotool

Doom Source files: <https://github.com/kilograham/rp2040-doom>

Download this file: <https://github.com/kilograham/rp2040-doom/archive/refs/heads/rp2.zip>

Installing SDL2 dependencies: <https://github.com/libsdl-org/SDL>

- 1) Download this file for SDL2 devel:
<https://github.com/libsdl-org/SDL/releases/download/release-2.30.10/SDL2-devel-2.30.10-mingw.zip>
- 2) Extract and navigate into the "x86_64-w64-mingw32" folder inside SDL2-devel; add address to user path
- 3) Add the user paths for the SDL2 "include" and "lib" directories
- 4) SDL2-mixer: https://github.com/libsdl-org/SDL_mixer
Download this file:
https://github.com/libsdl-org/SDL_mixer/releases/download/release-2.8.0/SDL2_mixer-devel-2.8.0-mingw.zip
- 5) From 7zip, extract SDL_mixer into the SDL2-devel "include" directory
- 6) From 7zip, extract the 3 lib files into the SDL2-devel "lib" directory
- 7) SDL2-net: https://github.com/libsdl-org/SDL_net
Download this file:

https://github.com/libSDL-org/SDL_net/releases/download/release-2.2.0/SDL2_net-devel-2.2.0-mingw.zip

- 8) From 7zip, extract the 3 lib files into the SDL2-devel "lib" directory
- 9) From 7zip, extract SDL_mixer into the SDL2-devel "include" directory

Building Doom for RP2350:

- 1) Navigate into RP2350 arm build directory, insure it is empty.
- 2) Execute:
`cmake -DCMAKE_BUILD_TYPE=MinSizeRel -DPICO_BOARD=vgaboard
-DPICO_SDK_PATH=<path to your pico sdk> -DPICO_PLATFORM=rp2350-arm-s
-Dpicotool_DIR=<path to your picotool> -DPICO_EXTRAS_PATH=<path to your pico_extras> -G
"NMake Makefiles" ..`
- 3) Execute: `nmake`
- 4) Put the Pico 2 into Bootsel mode and install game engine by dragging "doom_tiny_usb.uf2" from the "src" directory of build directory into the Pico 2.
- 5) Unplug and reconnect the Pico 2 in Bootsel mode
- 6) Navigate into the root directory of Doom (where doom1.whx is)
- 7) Execute: `picotool load -v -t bin doom1.whx -o 0x10042000`.
- 8) Unplug the Pico 2 and plug it into a powered USB hub with keyboard attached

Building RP2350 RISC-V binary:

- 1) From inside the risc-v build directory, execute: `cmake -DPICO_PLATFORM=rp2350-riscv
-Dpicotool_DIR=F:\Users\tools\picotool
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master
-DPICO_TOOLCHAIN_PATH=F:\Users\tools\bin -G "NMake Makefiles" ..`
- 2) `nmake`

Raspberry Pi Pico SDK: <https://github.com/raspberrypi/pico-sdk>

example source code from my GitHub: <https://github.com/sladorsoft/Raspber...>

Some of the above tools are available in winget. Here are the commands for installing them:

ARM GCC toolchain: "winget install Arm.GnuArmEmbeddedToolchain"

Python: "winget install python3"

make for Windows: "winget install Gnuwin32.Make"

CMake: "winget install Kitware.CMake"

Visual Studio Code: "winget install Microsoft.VisualStudioCode"

```
cmake -DPICO_PLATFORM=rp2040 -DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master
-DSDL2_INCLUDE_DIR=F:\Program_Files_(x86)\SDL2\include
-DSDL2_LIBRARIES=F:\Program_Files_(x86)\SDL2\lib -G "NMake Makefiles" ..
```

```
cmake -DPICO_PLATFORM=rp2350-arm-s -Dpicotool_DIR=F:\Users\tools\picotool
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master -G "NMake Makefiles" ..
```

```
cmake -DPICO_PLATFORM=rp2350-riscv -Dpicotool_DIR=F:\Users\tools\picotool
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master
-DPICO_TOOLCHAIN_PATH=F:\Users\tools\bin -G "NMake Makefiles" ..
```

Rp2040 doom compile

```
cmake -DCMAKE_BUILD_TYPE=MinSizeRel -DPICO_BOARD=vgaboard
-DPICO_SDK_PATH=F:\Users\pico2\pico-sdk -DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master
-G "NMake Makefiles" ..
```

Compiling sdl2 (from a build directory in the source file directory):

```
cmake -DCMAKE_PREFIX_PATH=F:\Program_Files_(x86) -G "NMake Makefiles" ..
```

Or

```
cmake -DCMAKE_INSTALL_PREFIX=F:\Program_Files_(x86) -G "NMake Makefiles" ..
```

Set lib to F:\Program_Files_(x86)\SDL\lib

Trying doom for rp2350 with sdl2 called out

```
cmake -DPICO_PLATFORM=rp2350-arm-s -Dpicotool_DIR=F:\Users\tools\picotool
-DSDL2_INCLUDE_DIR=C:\Users\Owner\mingw_dev_lib\SDL2-2.30.10\x86_64-w64-mingw32\inc
lude
-DSDL2_LIBRARIES=C:\Users\Owner\mingw_dev_lib\SDL2-2.30.10\x86_64-w64-mingw32\lib
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master -G "NMake Makefiles" ..
```

```
cmake -DPICO_PLATFORM=rp2350-arm-s -Dpicotool_DIR=F:\Users\tools\picotool
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master -G "NMake Makefiles" ..
```

```
cmake -DCMAKE_BUILD_TYPE=MinSizeRel -DPICO_BOARD=vgaboard
-DPICO_SDK_PATH=F:\Users\pico2\pico-sdk -DPICO_PLATFORM=rp2350-arm-s
-Dpicotool_DIR=F:\Users\tools\picotool\picotool
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master -G "NMake Makefiles" ..
```

Without picotool:

```
cmake -DCMAKE_BUILD_TYPE=MinSizeRel -DPICO_BOARD=vgaboard
-DPICO_SDK_PATH=F:\Users\pico2\pico-sdk -DPICO_PLATFORM=rp2350-arm-s
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master -G "NMake Makefiles" ..
```

Building and installing picotool:

(<https://github.com/raspberrypi/pico-sdk/pull/1820>)

In short, you can run the following CMake commands from your picotool build directory to install the required files to \$MY_INSTALL_DIR (replacing \$MY_INSTALL_DIR with the path you want to use)

```
cmake -DCMAKE_INSTALL_PREFIX=F:\Users\tools -DPICOTOOL_FLAT_INSTALL=1 ..
```

```
cmake --build .
```

```
cmake --install .
```

```
cmake -DCMAKE_BUILD_TYPE=MinSizeRel -DPICO_BOARD=vgaboard
```

```
-DPICO_SDK_PATH=F:\Users\pico2\pico-sdk -DPICO_PLATFORM=rp2350-arm-s
```

```
-Dpicotool_DIR=F:\Users\tools\picotool
```

```
-DPICO_EXTRAS_PATH=F:\Users\pico2\pico-extras-master -G "NMake Makefiles" ..
```